Group 5 Project Proposal : Boolean Satisfiability (SAT)

Kimchour LUY (2025136) Soksovan HENG (2025427)
Singyang HOEURNG (2025138) Monymatin MOENG (2025385)
Sophapor HENG (2025530)

2026-10-30

Title

Exploring rudimentary boolean satisfiability (SAT) solvers

Problem

To define the SAT problem, we first define the boolean set $\mathbb{B} = \{0, 1\}$.

The SAT problem states, given a boolean function $f : \mathbb{B}^n \to \mathbb{B}$, where the function f only uses the disjunction operator $\vee$, conjunction operator $\wedge$, and the negation operator $\neg$; we want to determine whether:

$$\exists \mathbf{x} \in \mathbb{B}^n \text{ such that } f(\mathbf{x}) = 1$$

If this returns a true, we say that the function f is satisfiable, otherwise, it's unsatisifiable.

Example

Suppose that we define a boolean function:

$$f(x_1, x_2, x_3) = x_1 \wedge (x_2 \vee \neg x_3)$$

Notice that when we set $x_1 = 1$, $x_2 = 1$, $x_3 = 0$, that is:

$$f(1, 1, 0) = 1 \wedge (1 \vee \neg 0) = 1 \vee 1 = 1$$

Thus, we say that f is satisfiable, because there exists an assignment $\mathbf{x} = (1, 1, 0)$, which causes $f(\mathbf{x}) = 1$.

Importance

SAT is the canonical NP-complete problem, and because every problem that is NP-complete is equivalent to any other problem that is also NP-complete, we can reduce all NP-complete problems into SAT, and then use a generic blackbox SAT solver to find the solution to the original problem. In that sense, a SAT solver can be thought of as a general solver which works well for some NP-complete problems, especially if the solver is a state-of-the-art solver.

In the theoretical world, SAT is important, mainly because it was the first ever problem proven to be NP-complete (by the Cook-Levin theorem). If we can reduce the boolean satisfiability problem into another problem in NP, we would then prove that that problem is NP-complete, thus, proving NP-completeness turns into a problem in reduction.

Solution

The simplest method of solving the boolean satisfiability problem is simply by constructing a truth table. This naive method requires that you check all 2^n possible assignments for the vector $\mathbf{x} \in \mathbb{B}^n$ in the worst case, thus, it's an exponential time algorithm.

A more well-informed approach would be to take note of the structure of f, and then use some techniques to simplify the problem of finding a truth assignment of f. The asymptotic worst case scenario would still be exponential in time (because it's probably possible to construct some boolean expression f where it is hard to simplify further), however, in practice, these heuristics would speed up our search significantly.

In our case, we will try to implement the Davis-Putnam-Logemann-Loveland (DPLL) in order to build a rudimentary SAT solver.

Abstract

SAT's usefulness is amplified when viewed from the lens of different problems, thus, we want first motivate the SAT problem by expressing some interesting problems into a problem in SAT. We will then construct different implementations of SAT solvers and gauge their performance using some dataset, and compare it to state-of-the-art solvers. Modern SAT solvers accept boolean functions in the Conjunctive Normal Form (CNF), written in a format called the DIMACS CNF format, thus, we will also write a simple DIMACS CNF parser and constructor.